WEX 428 - NLP / Sentiment Analysis Project Report

Fake News Explainability Platform

An AI-powered web application that classifies news articles as Fake or Real and explains the decision with LIME and SHAP

Student	[Student name]
Student ID	[Student ID]
Instructor	[Instructor]
University	Ostim Technical University
Date	18 October 2026
Source code	[GitHub repository link]

Every figure in this report was produced by the code in the repository: experiment results by *scripts/run_experiments.py*, platform metrics by the administrator training pipeline, and screenshots by an automated browser test.

1. Introduction

Most fake-news detectors return a bare label. This project builds a platform that also shows **why** an article was classified as it was: the words that pushed the decision are marked inside the article, with a confidence score, keywords and the five most similar articles from the dataset. It is explicitly a **decision-support tool**, not a truth detector. Following the course rules, no ready-made LLM API is used; every model is trained locally.

Objectives. (1) a complete NLP pipeline with documented experiments; (2) at least three compared models; (3) explainability with LIME / SHAP; (4) a full-stack application with separate user and administrator interfaces, JWT security, a relational database and a REST API; (5) reproducible deployment.

2. Dataset

The provided dataset has two raw files, *Fake.csv* and *True.csv*, with the columns title, text, subject and date (the label is implied by the file name; we add *label*: 0 = Fake, 1 = Real). Together they hold 44,898 rows.

Stage	Articles	Fake	Real
Raw files	44,898	23,481	21,417
After removing empty and duplicate articles (experiments)	38,653	17,459	21,194
Imported by the platform (also rejects titles / subjects that are empty and texts under 5 words)	38,559	17,365	21,194

Data-quality findings. Fake.csv contains 6,026 repeated texts and 630 empty ones; True.csv contains 225 repeated texts. In the platform import, Fake.csv produced 17,365 valid articles (773 rejected, 5,343 duplicates ignored) and True.csv 21,194 (1 rejected, 222 duplicates). The instructor's *train.csv* and *test.csv* share 82 identical articles, so we built our own stratified 80/20 split (seed 42) after de-duplication.

Label leakage. The token *reuter* occurs in 99.8% of Real articles and 1.2% of Fake ones, and the *subject* column separates the classes perfectly. We therefore never use *subject* as a feature and added a source-artifact removal step to the pipeline (section 3). Details: docs/leakage_analysis.md.

3. NLP pipeline

The same code runs for training and for user predictions (one preprocessing module), so the two cannot drift apart.

- **Cleaning:** HTML entity / tag removal, URL removal, e-mail removal, number removal, apostrophe and punctuation removal, lowercasing, whitespace normalisation.
- **Source-artifact removal (added):** the Reuters dateline and the word 'Reuters', plus publisher boilerplate such as 'Featured image via', 'Getty Images', '(VIDEO)'.
- **Tokenisation and stop-words:** whitespace tokens, 318-word editable stop-word list (administrators can change it); tokens shorter than 2 letters dropped.
- **Stemming:** Snowball (English) stemmer with a cache.
- **Features:** TF-IDF with unigrams and bigrams, at most 50,000 features (10,000 for tree models), min_df 3, max_df 0.9, sub-linear term frequency.
- **Split:** 80% train / 20% held-out test, stratified, seed 42. The test set is used only for evaluation.

Classifiers. Logistic Regression (C = 4), linear SVM (C = 0.5, wrapped in sigmoid calibration so it outputs probabilities), multinomial Naive Bayes (alpha = 0.1), Random Forest (100 trees) and XGBoost (200 trees, depth 6). The confidence score is the predicted-class probability.

4. Experiments

All experiments use one split (30,922 train / 7,731 test articles). The standard error of an accuracy near 98.6% on 7,731 articles is about 0.13 points, so differences smaller than roughly 0.3 points are **not** evidence that one setting is better.

4.1 Preprocessing strategies and label leakage

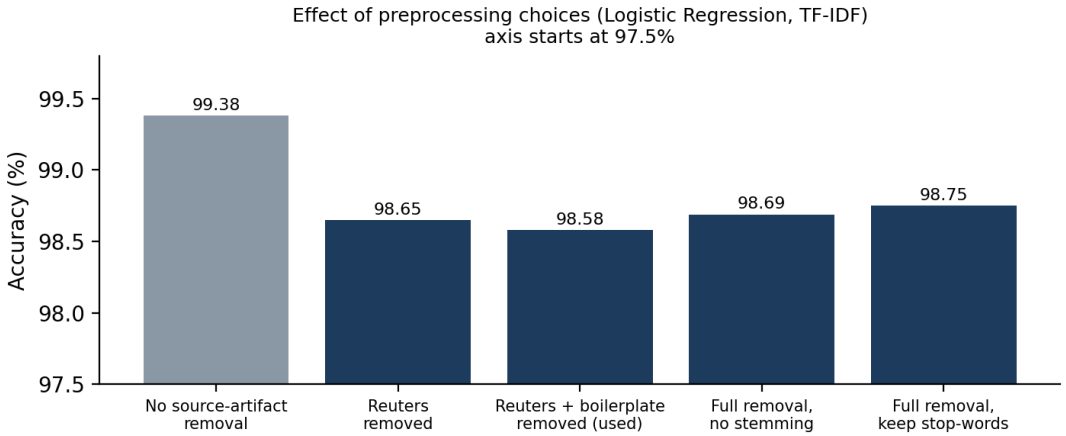


Figure 1. Removing the Reuters fingerprint lowers accuracy by about 0.7 points; stemming and stop-word choices change it by less than 0.2 points (within noise).

Variant	Accuracy	F1
L0_no_leakage_removal	99.38%	99.44%
L1_reuters_removed	98.65%	98.78%
L2_full_removal	98.58%	98.71%
L2_no_stemming	98.69%	98.81%
L2_keep_stopwords	98.75%	98.86%

On the instructor's own split, as given, Logistic Regression reaches 99.12%; deleting only the token *reuter* gives 98.60%. The drop is small because the classes also differ in writing style, not only in that one word (section 6).

4.2 Feature extraction

Feature extraction	Accuracy	F1	Training time
Bag of Words (unigrams)	97.96%	98.15%	8.83 s
TF-IDF (unigrams)	98.58%	98.71%	5.44 s
TF-IDF (unigrams + bigrams)	98.72%	98.84%	28.09 s

Bag of Words is clearly weaker. TF-IDF with bigrams is the best by 0.14 points over unigrams, which is inside the noise, and trains about five times slower. We kept TF-IDF with bigrams because it is the best observed configuration and the cost is acceptable; this choice is justified by the experiment but not by a significant margin. Word2Vec, FastText and trigrams were not tested.

4.3 Model comparison (experiment split)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Train time
Logistic Regression	98.72%	98.25%	99.43%	98.84%	0.9991	28.61 s
Naive Bayes	95.36%	95.86%	95.66%	95.76%	0.9879	26.38 s
Linear SVM (calibrated)	98.97%	98.87%	99.25%	99.06%	0.9993	28.26 s
Random Forest	97.43%	96.98%	98.37%	97.67%	0.9974	86.81 s
XGBoost	98.58%	98.34%	99.08%	98.71%	0.9990	239.21 s

5. Final models trained inside the platform

The administrator pipeline trained all five algorithms on the imported database (38,559 articles; 30,847 train / 7,712 test) and recorded the metrics below. They differ slightly from section 4.3 because the platform import also rejects very short articles, so the two data sets are not identical.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Train time
Logistic Regression (ACTIVE)	98.76%	98.32%	99.43%	98.87%	0.9993	46 s
Linear SVM (calibrated)	98.95%	98.89%	99.20%	99.05%	0.9994	32 s
Naive Bayes	95.44%	96.13%	95.54%	95.84%	0.9889	27 s
Random Forest	97.63%	97.16%	98.56%	97.86%	0.9977	105 s
XGBoost	98.70%	98.52%	99.13%	98.82%	0.9989	251 s

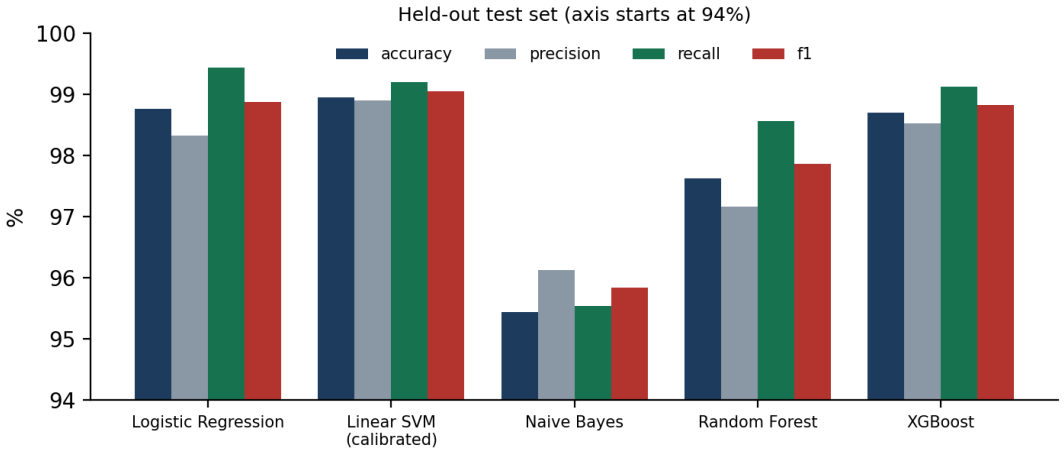


Figure 2. Metrics of the five models on the held-out test set (platform training).

Active model: Logistic Regression. The linear SVM scores 0.19 points higher in accuracy, which is within the noise of a single split. Logistic Regression is kept active because it supports exact SHAP values, trains in under 30 seconds and gives well-behaved probabilities. Naive Bayes and Random Forest are clearly weaker.

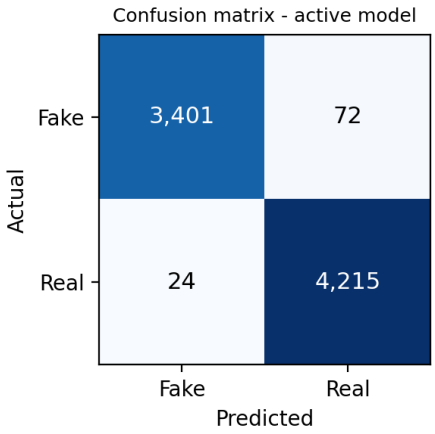


Figure 3. Confusion matrix of the active model on the test set.

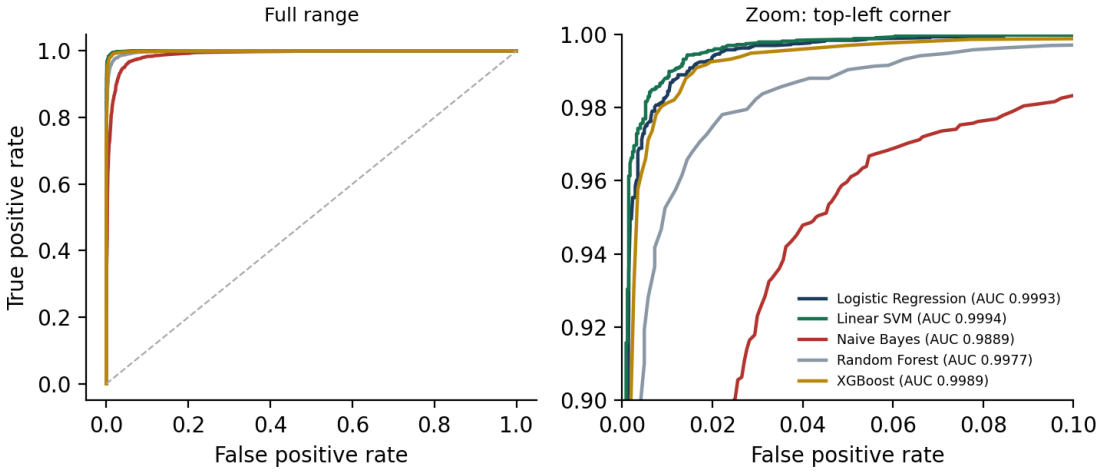


Figure 4. ROC curves of the five models on the test set. The strong models are almost indistinguishable; Naive Bayes and Random Forest sit visibly lower in the zoomed corner.

Important caveat. Both classes come from only two publishers. High accuracy shows that the model recognises the style of these sources; it does not show that it can verify claims or generalise to other outlets. A confident mistake exists in our held-out demo set (a Fake article classified Real with 98.4% confidence), which is why the interface always shows the confidence and the evidence.

6. Explainability

LIME (any model): the article is perturbed by removing random words; a local linear surrogate is fitted to the model's responses (400 samples) and its weights are the explanation. Words that preprocessing removes (stop-words, one- or two-letter fragments, numbers) cannot influence the model, so their small surrogate weights are discarded - an issue we found when reviewing the first screenshots. **SHAP** (Logistic Regression and XGBoost): exact linear SHAP values on the TF-IDF features against the training-set mean, mapped back from stems to the readable words of the article.

Sign convention. A positive weight pushes towards the predicted class, a negative weight against it. In the article view, red marks push towards Fake and green marks towards Real; darker means stronger.

What the explanations reveal. In this example 'said' is the strongest word against Fake (reporting language typical of the Real class), while 'read' (from 'Read more') and informal wording such as 'didn' push towards Fake. This agrees with the leakage analysis: the model relies heavily on *how* an article is written. The Administrator's explainability page aggregates such words across all

predictions so that this kind of bias can be monitored.

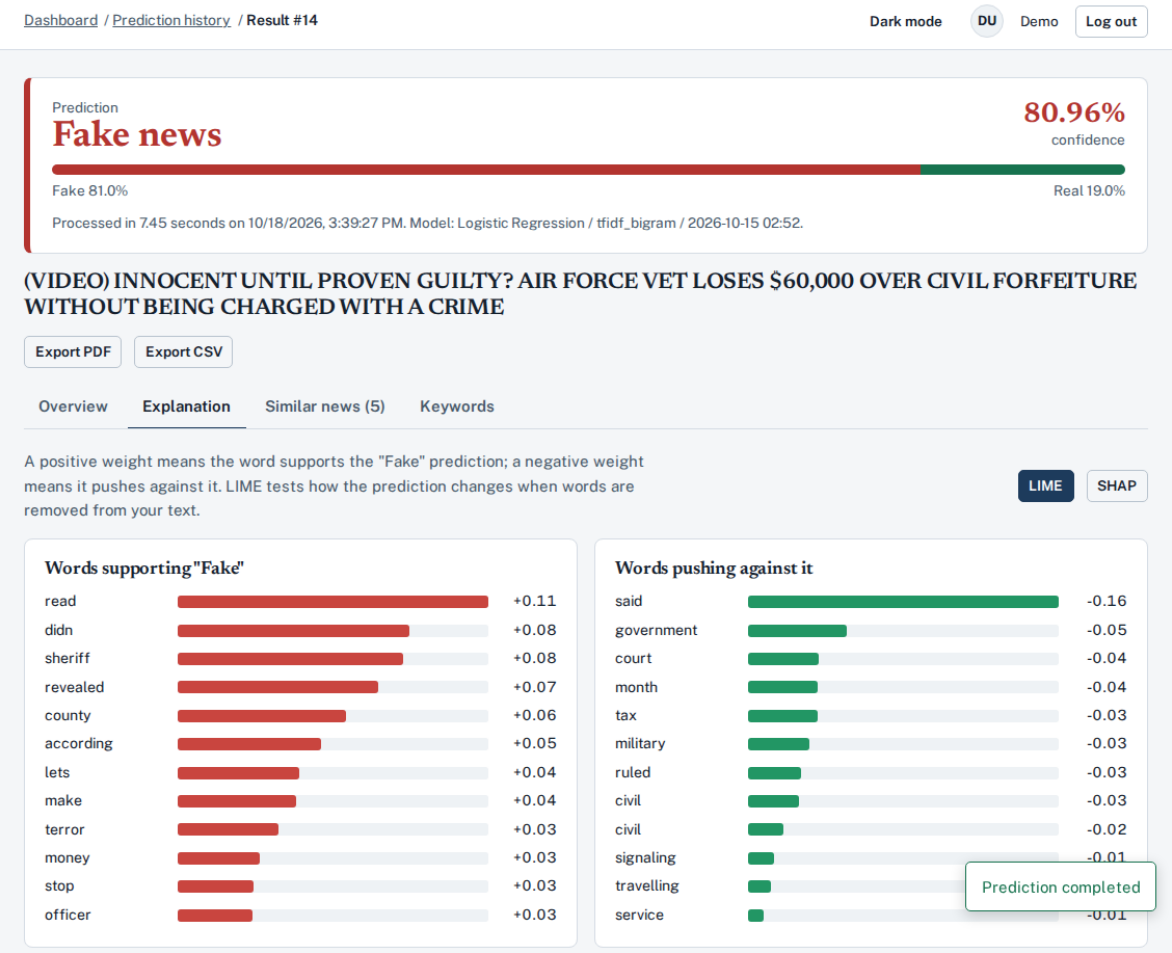


Figure 5. Explanation page for a held-out Fake article (80.96% confidence): supporting and opposing words and the importance ranking.

7. System design and implementation

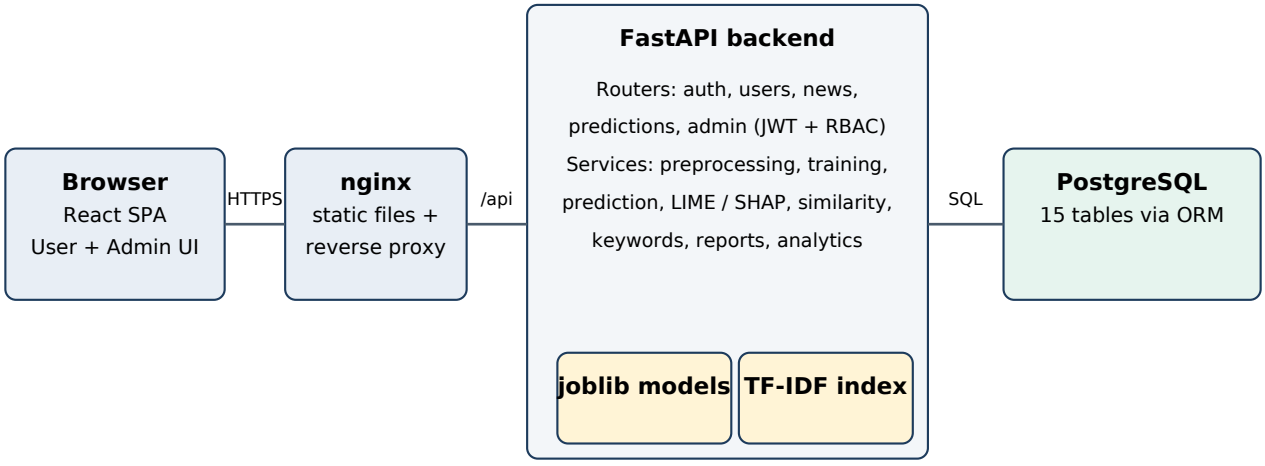


Figure 6. Architecture.

Backend. FastAPI with a layered design (routers, services, ORM models). 65 REST operations cover authentication, profile, analysis, history, explanations, similar news, exports and the whole administration area; all 15 endpoints listed in the assignment exist. Long jobs (dataset import, model training) run in background threads and report progress through the database.

Database. PostgreSQL through SQLAlchemy. The ten required tables (Users, NewsArticles, PredictionHistory, Keywords, ExplainabilityResults, SimilarNews, Models, DatasetImports, SystemLogs, UserSettings) are implemented with foreign keys and the requested indexes, plus five support tables (training runs, revoked tokens, password-reset tokens, application settings, keyword dictionary). 'Only one active model' is enforced by a partial unique index in the database, not only in code. Duplicate articles are prevented by a unique content hash.

Security. bcrypt hashing (with a SHA-256 pre-hash), signed JWTs with revocation on logout, role checks read from the database on every admin request, login throttling, parameterised queries only, upload validation by extension, size and file signature, secrets from environment variables, non-root container, security headers, and no raw-HTML rendering in the React app (XSS).

Frontend. React with two separate interfaces (users and administrators), responsive layout, light and dark themes, breadcrumbs, sidebar, search, pagination, confirmation dialogs, loading states and 403 / 404 / 500 pages.

Deployment. Docker Compose (PostgreSQL, backend, nginx serving the built frontend), a bootstrap script that imports the dataset and trains the models, and a GitHub Actions workflow.

Area	Endpoints (examples)
Authentication	POST /api/auth/register, /login, /logout, /forgot-password, /reset-password; GET /api/auth/me
User	GET/PUT /api/users/profile, PUT /password, POST /profile-image, GET/PUT /settings, GET /dashboard
Analysis	POST /api/news/analyze, /api/news/upload; GET /api/news/similar/{id}
History	GET /api/predictions/history, /{id}, /{id}/export?format=pdf|csv; DELETE; GET /api/explanations/{id}
Administration	users, datasets (preview / upload / progress), news, subjects, models (train / retrain / activate / delete), analytics, explainability, stop-words, settings, logs

8. Testing and defects found

Testing used three layers: **22 API tests** (registration and validation, password hashing, JWT revocation, role access, password reset, dataset validation and duplicates, training and single-active-model rule, the full analysis workflow, user isolation, exports, file upload checks, administration), run on **SQLite and on PostgreSQL 16**; an automated **browser test** in headless Chromium that walks through both interfaces; and manual review of the screenshots. All pass. The tests also found real defects, all fixed:

Defect	How it was found	Fix
PostgreSQL rejected numpy float types stored for keyword scores (SQLite accepted them silently)	Running the API suite on PostgreSQL	Explicit float() / int() conversion
After training, the API returned a stale model status	API test	Reload the model from the database after the background job
Route /users/{id}/{action} swallowed /users/{id}/reset-password	API test	Separate explicit routes
All charts crashed the page (Chart.js controllers not registered); the error page itself crashed outside the router	Browser test	Register controllers; plain link in the error page
Top bar overflowed on phones	Browser test (390 px viewport)	Hide name and avatar, shorten breadcrumbs
LIME gave noise weights to stop-words that the pipeline removes	Screenshot review	Filter words that preprocessing discards
Training split order was not fixed (could differ on PostgreSQL)	Code review	Order articles by id before splitting

Defect	How it was found	Fix
Predictions ignored the pasted headline although the models are trained on 'title. text' (81.0% vs 80.65% for the same article)	Comparing the UI result with the demo-article script	Same document form at inference; regression test

Not tested: the Docker images were written but not built in the development environment; performance under many concurrent users; accessibility audit with assistive technology.

9. Coverage of the assignment requirements

Requirement	Status	Notes
Preprocessing, tokenisation, stop-words, stemming (6.1)	Done	plus source-artifact removal
Feature extraction, justified by experiment (6.2)	Done	BoW, TF-IDF, TF-IDF + bigrams compared; Word2Vec / FastText / trigrams not tested
At least three models compared (6.3)	Done	five models; accuracy, precision, recall, F1, ROC-AUC, ROC curves and confusion matrix
LIME / SHAP, positive / negative words, highlighting (FR-5)	Done	LIME for all models; SHAP for Logistic Regression and XGBoost
Similar news top 5, keywords, prediction history, reports (FR-6, 7, 8, 11)	Done	TF-IDF + cosine; TF-IDF or RAKE keywords; PDF and CSV exports
Authentication: register, login, logout, forgot / reset password, JWT, RBAC (FR-1)	Done	reset link goes to the server log unless SMTP is configured; refresh tokens (optional) not implemented
Dataset import with validation, duplicates, progress, versions (FR-2, 8.3)	Done	training can be restricted to one dataset (Settings)
Model management: train, retrain, compare, activate, delete, history (FR-10)	Done	single active model enforced by the database
Administrator dashboard, analytics, explainability monitoring, logs (FR-9, 7.2)	Done	Chart.js charts
User dashboard, profile, preferences, delete account	Done	export-format preference is used in the history; language and e-mail-notification preferences are stored but not used yet
Light / dark mode, responsive, 404 / 500 pages	Done	the administrator's default theme applies to visitors who have not chosen one
Docker, Docker Compose, CI (9)	Written, not built	no Docker in the development environment; cloud deployment (AWS, Azure, GCP) not done
TXT / PDF upload (FR-3)	Done	scanned PDFs without text cannot be read

10. Limitations, ethics and future work

- **Source bias.** Two publishers, very different styles: the model learns style, so accuracy on other outlets is unknown and likely lower. Evaluating on an independent dataset is the most important next step.
- **Not a fact-checker.** The system never verifies a claim against evidence. Satire, opinion pieces and breaking news could be mislabelled. The interface warns users and shows low-confidence results explicitly.
- **Confident errors happen.** Users should read the highlighted evidence rather than trust the percentage alone.

- **Privacy.** Submitted articles are stored in each user's history; users can delete entries or their whole account.
- **Future work.** Cross-dataset evaluation, word embeddings or a fine-tuned transformer, claim-level explanations, multi-language support, and cloud deployment.

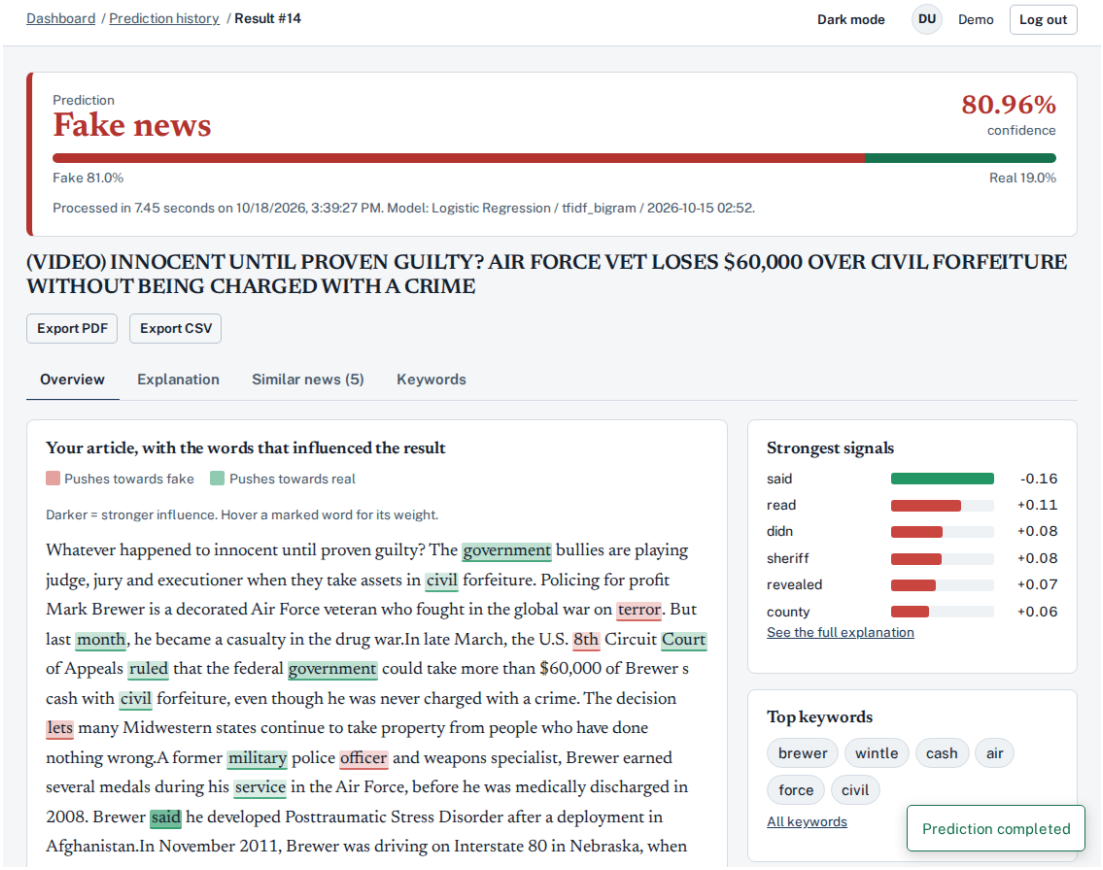
11. Conclusion

The project delivers a working, tested platform: a five-model comparison on 38,559 articles reaching up to 98.9% accuracy, explanations that show the evidence inside the article, and a secure two-interface web application. The experiments also show that high accuracy on this dataset partly reflects source style, and the report states that limitation openly instead of presenting the accuracy as proof of truth detection.

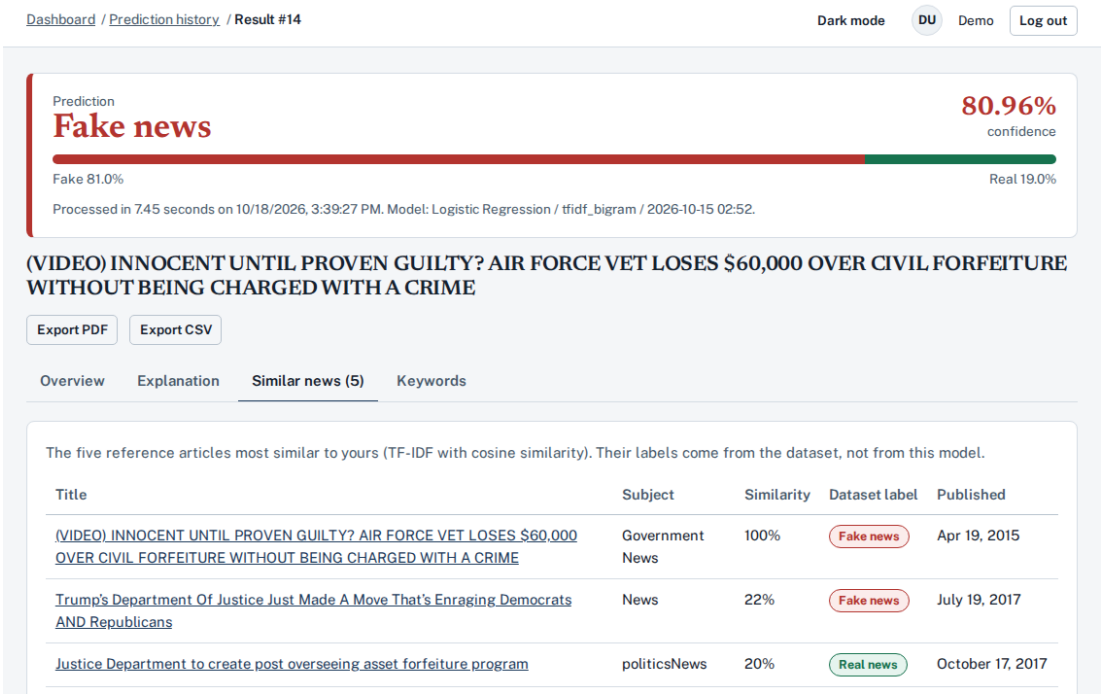
Appendix A. Screens of the application



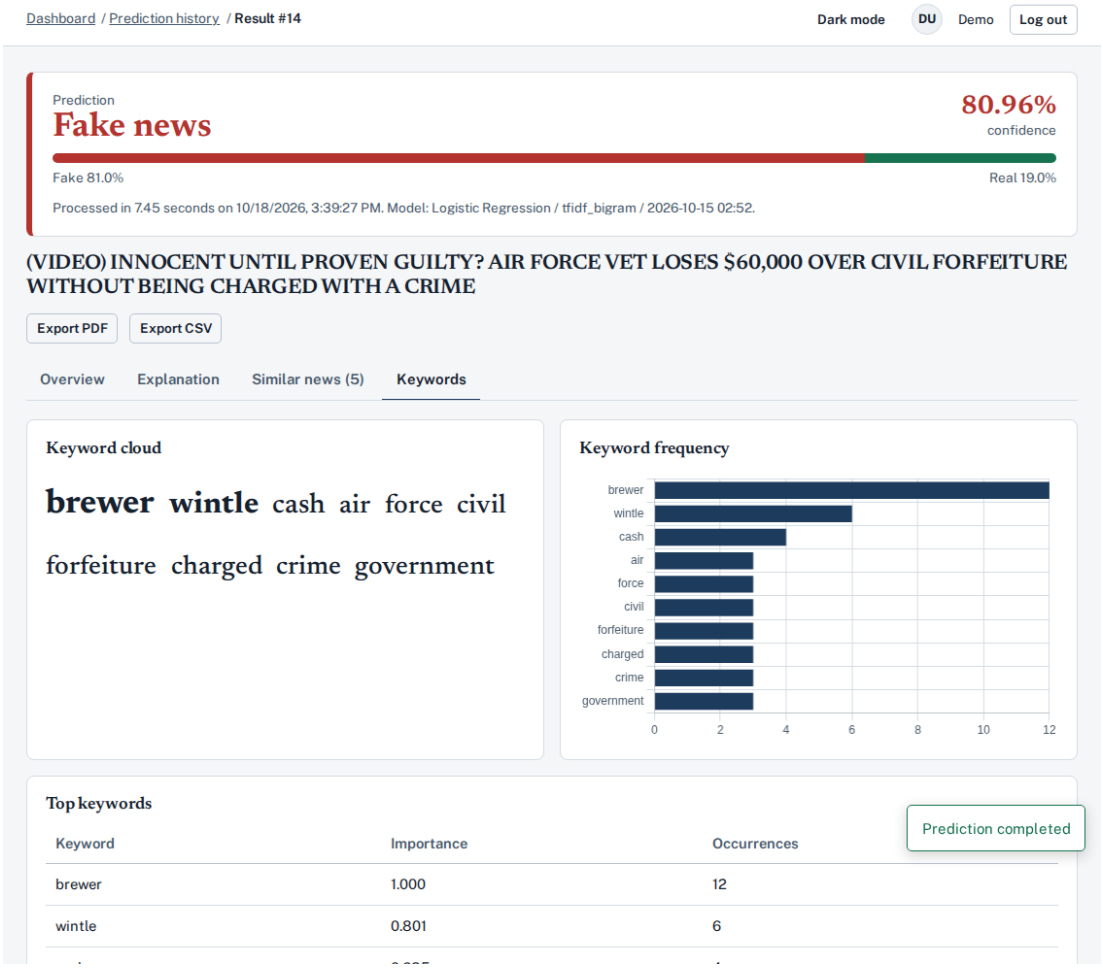
A.1 Analyze page: paste text or upload a document, with character and word counters.



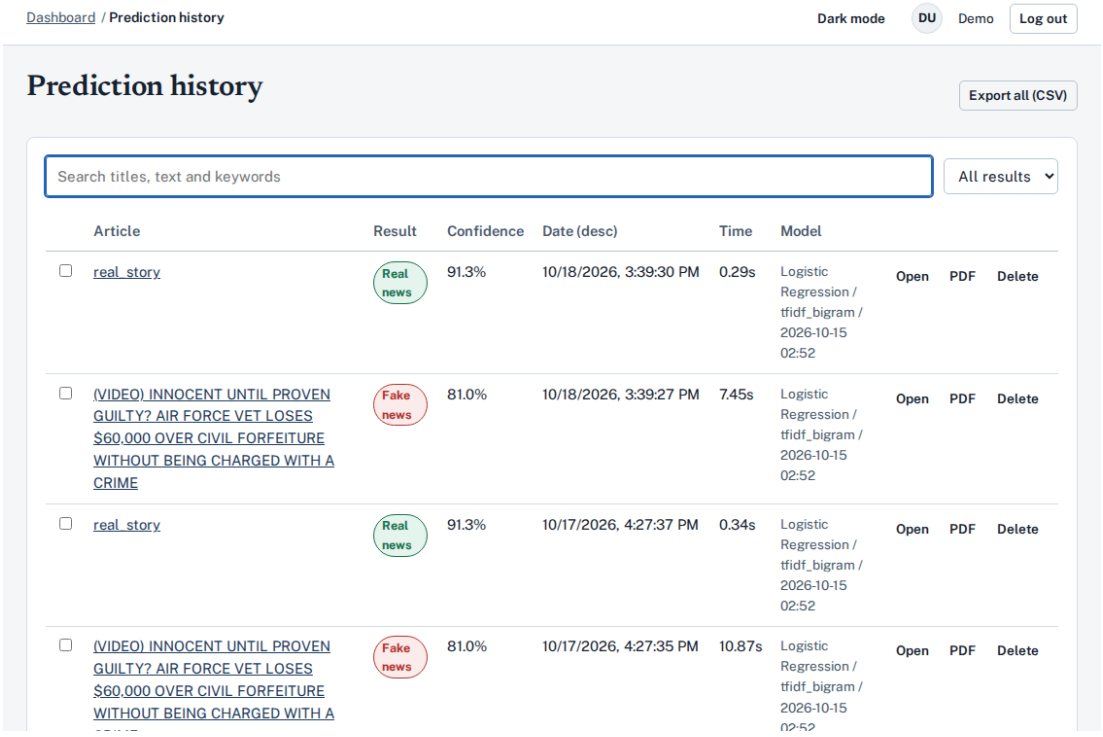
A.2 Result overview: verdict, confidence, the marked article, strongest signals and keywords.



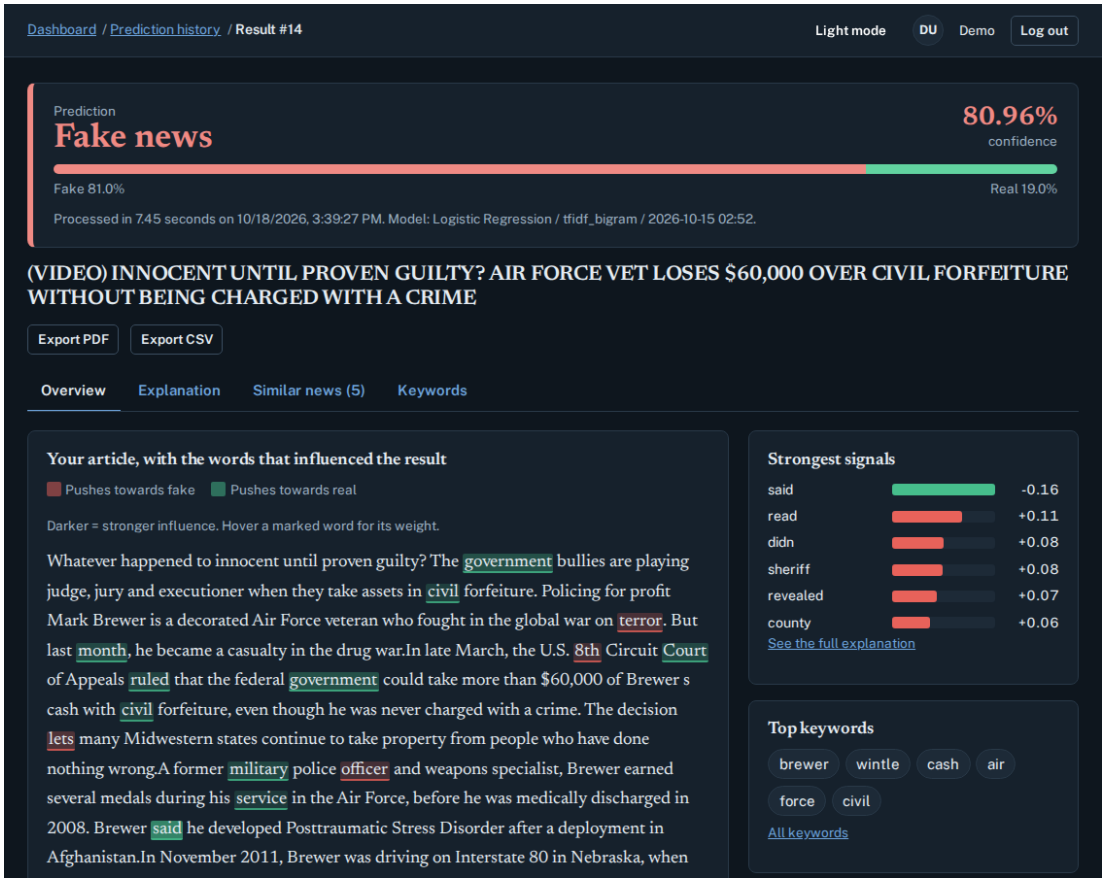
A.3 Similar news: the five closest dataset articles with similarity and dataset label.



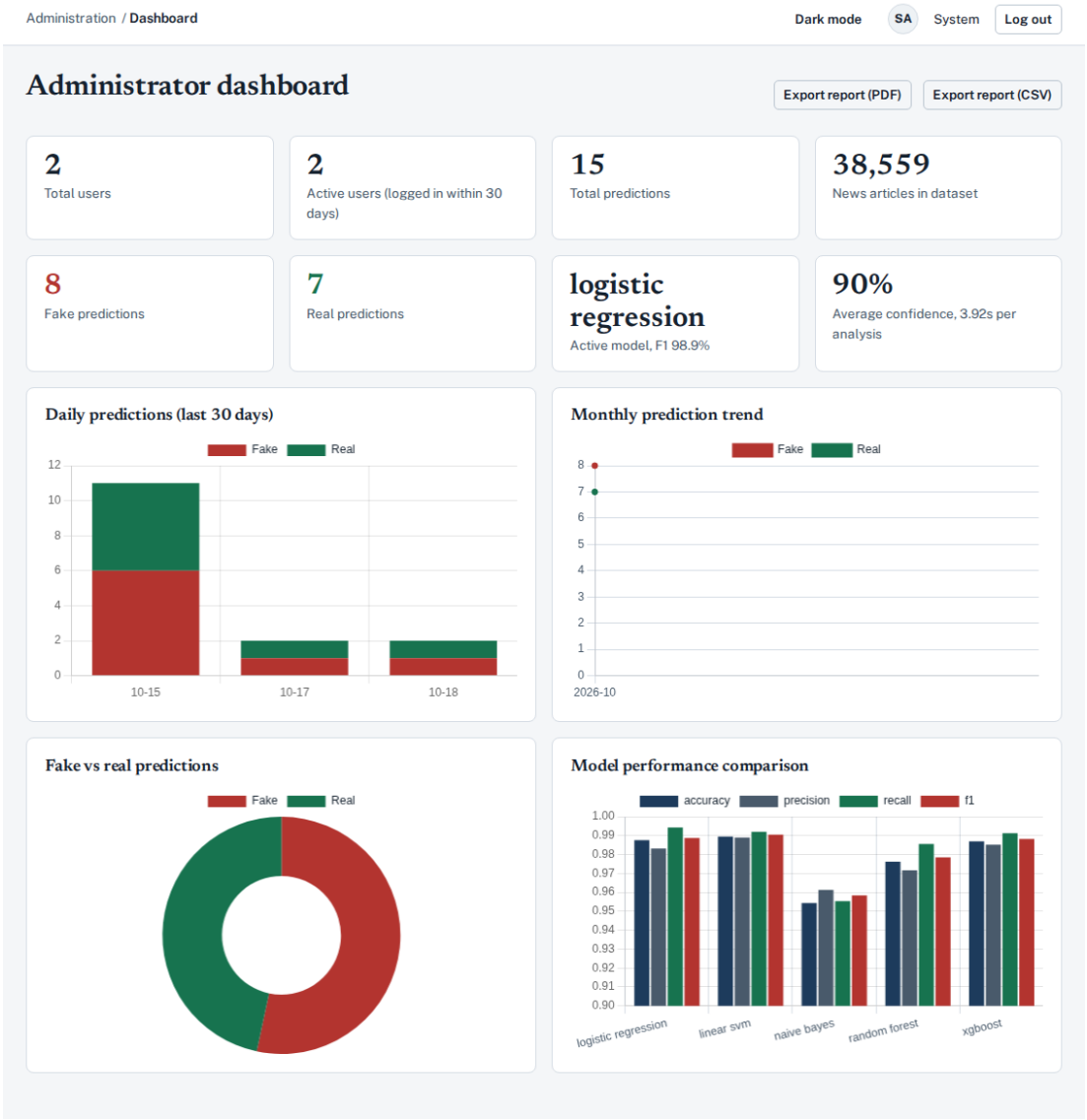
A.4 Keywords: cloud, frequency chart and ranked list.



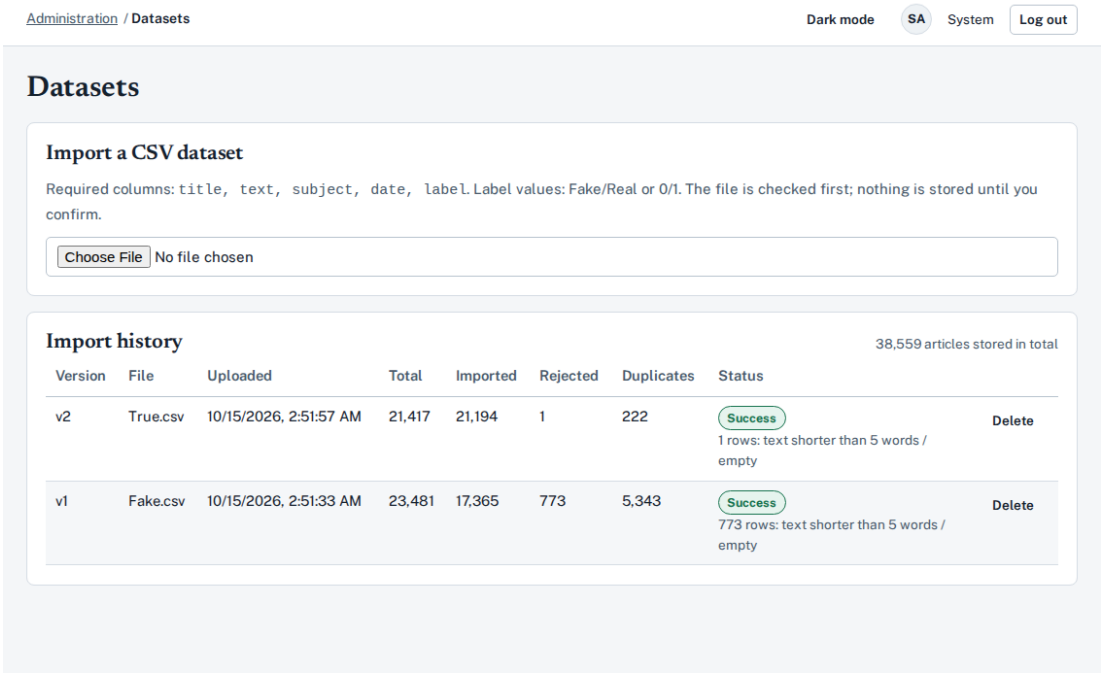
A.5 Prediction history with search, filters, sorting, bulk actions and exports.


Dashboard / Prediction history / Result #14
Light mode
DU
Demo
Log out
Prediction
Fake news
80.96%
confidence
Fake 81.0%
Real 19.0%
Processed in 7.45 seconds on 10/18/2026, 3:39:27 PM. Model: Logistic Regression / tfidf_bigram / 2026-10-15 02:52.
(VIDEO) INNOCENT UNTIL PROVEN GUILTY? AIR FORCE VET LOSES $60,000 OVER CIVIL FORFEITURE WITHOUT BEING CHARGED WITH A CRIME
Export PDF
Export CSV
Overview
Explanation
Similar news (5)
Keywords
Your article, with the words that influenced the result
Pushes towards fake
Pushes towards real
Darker = stronger influence. Hover a marked word for its weight.
Whatever happened to innocent until proven guilty? The government bullies are playing judge, jury and executioner when they take assets in civil forfeiture. Policing for profit Mark Brewer is a decorated Air Force veteran who fought in the global war on terror. But last month, he became a casualty in the drug war.In late March, the U.S. 8th Circuit Court of Appeals ruled that the federal government could take more than $60,000 of Brewer s cash with civil forfeiture, even though he was never charged with a crime. The decision lets many Midwestern states continue to take property from people who have done nothing wrong.A former military police officer and weapons specialist, Brewer earned several medals during his service in the Air Force, before he was medically discharged in 2008. Brewer said he developed Posttraumatic Stress Disorder after a deployment in Afghanistan.In November 2011, Brewer was driving on Interstate 80 in Nebraska, when
Strongest signals
said -0.16
read +0.11
didn +0.08
sheriff +0.08
revealed +0.07
county +0.06
See the full explanation
Top keywords
brewer
wintle
cash
air
force
civil
All keywords

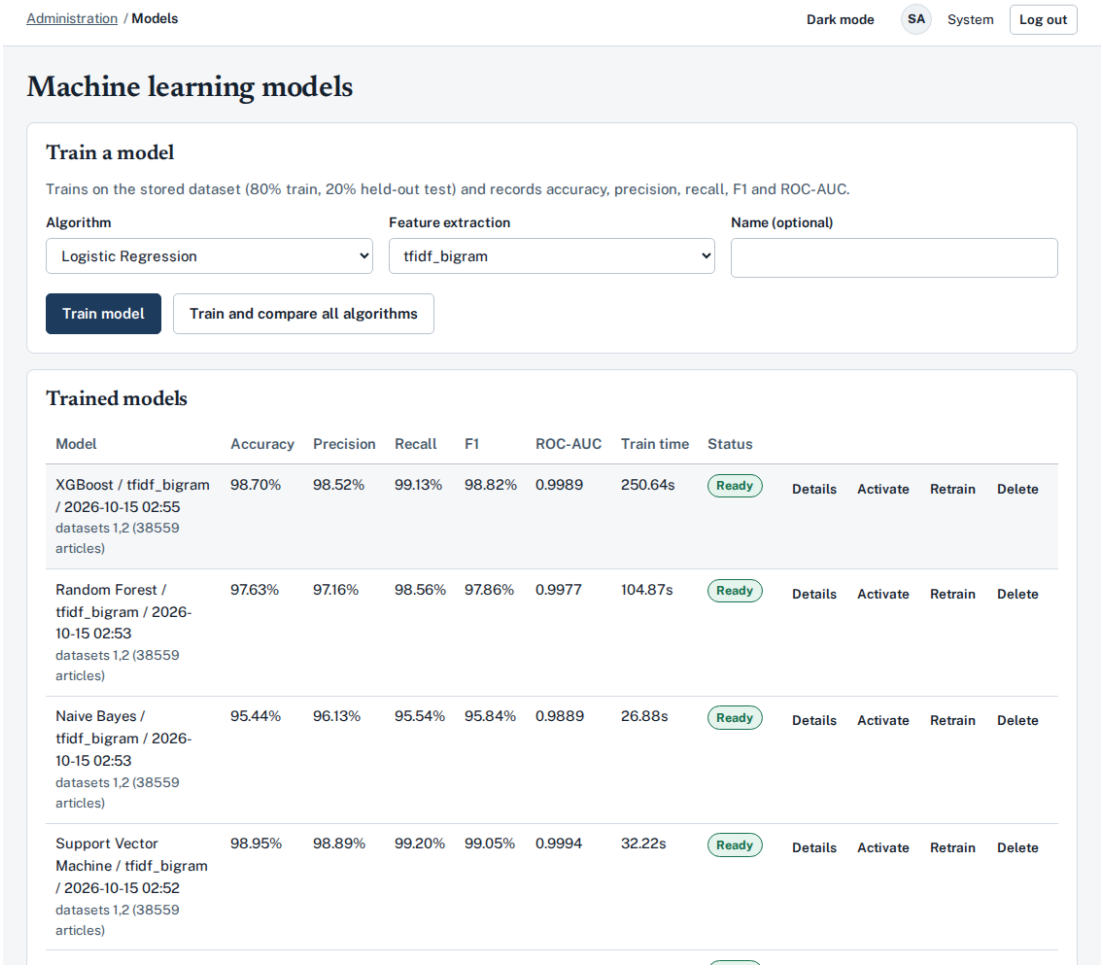
A.6 Dark mode.



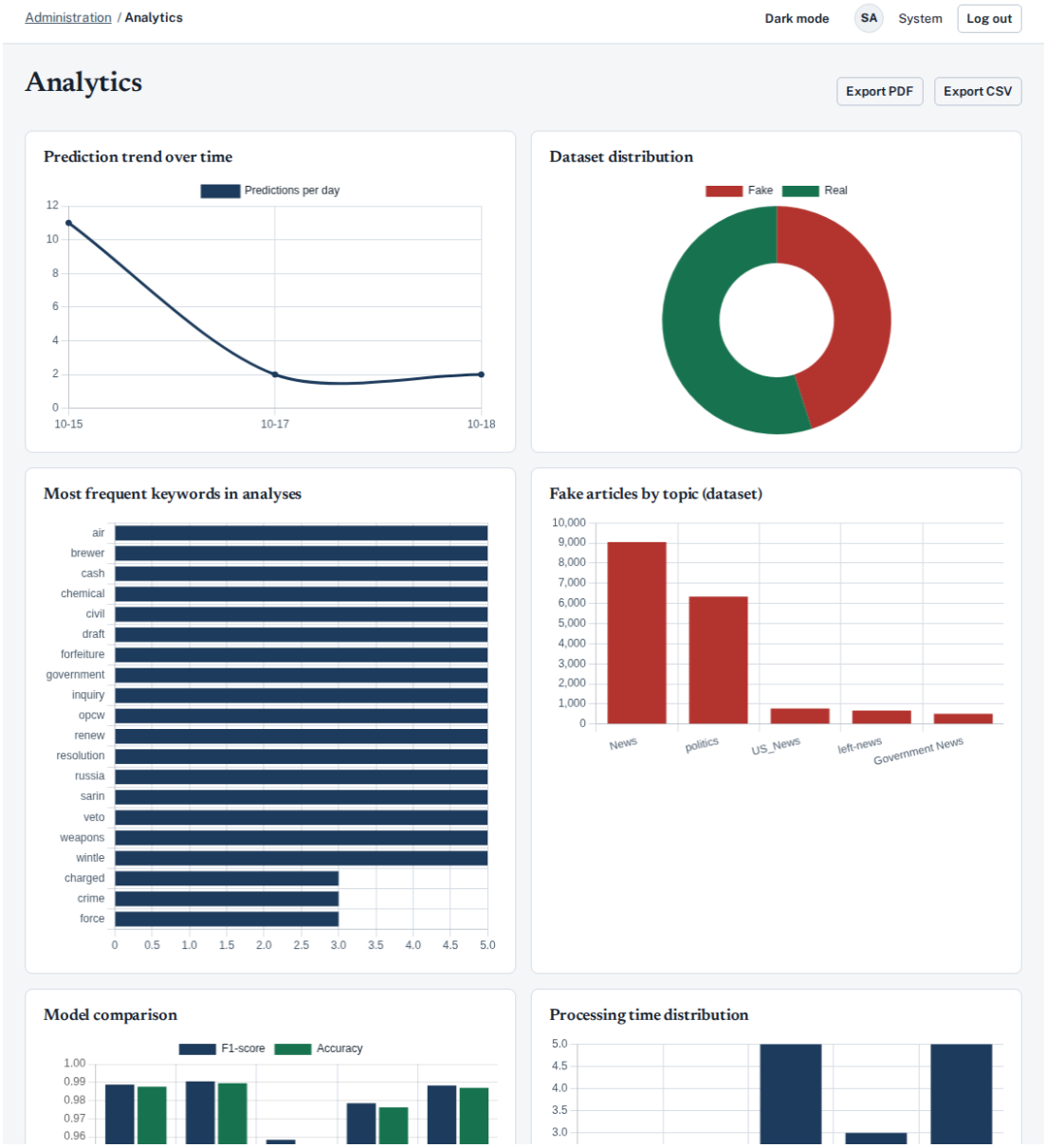
A.7 Administrator dashboard.



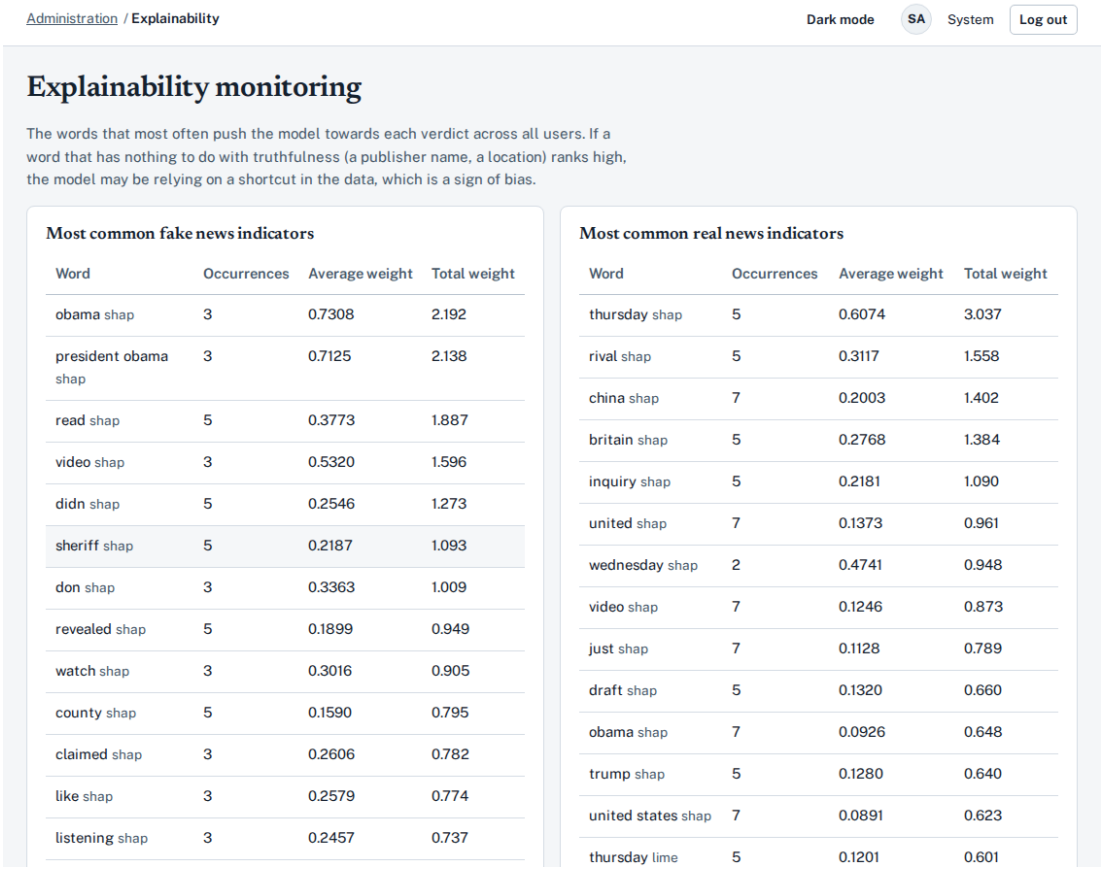
A.8 Dataset import history.



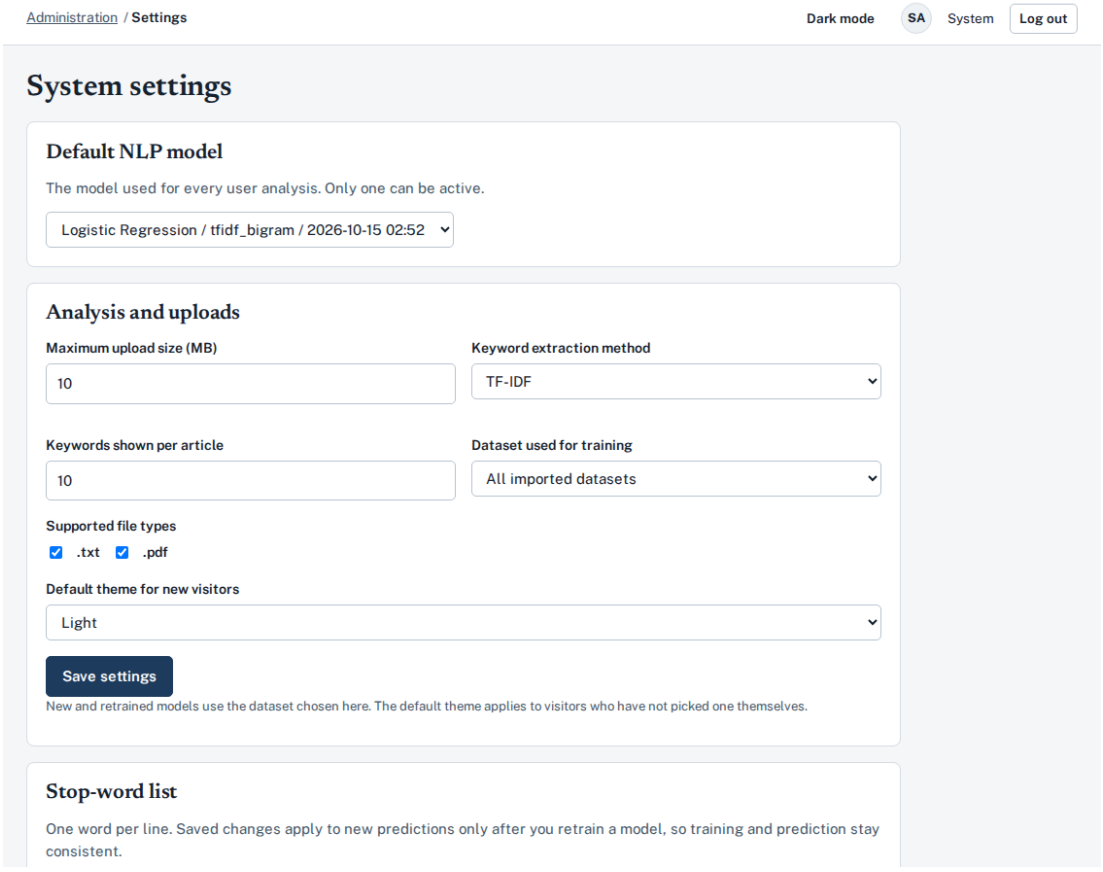
A.9 Model management and comparison.



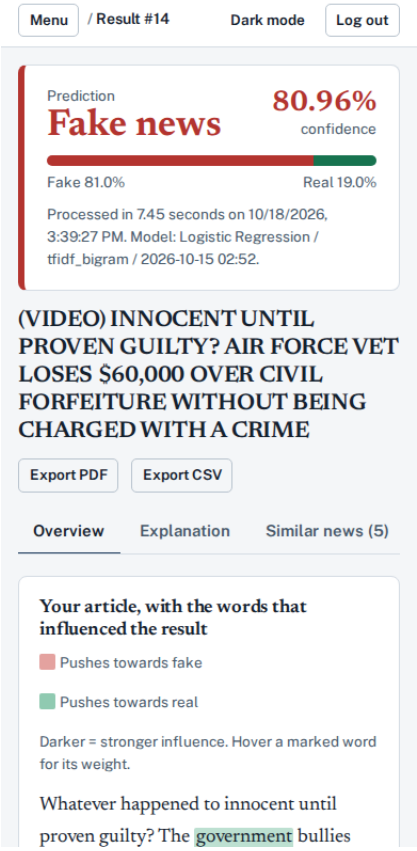
A.10 Analytics.



A.11 Explainability monitoring.



A.12 Settings, stop-words and keyword dictionary.



A.13 The result page on a phone-sized screen (390 px wide).

Source code: [GitHub repository link]

Run it: see README.md (Docker Compose or local development).